

FastCAE-Abaqus 集成示例

（插件拓展集成）

FastCAE 研发小组

2020 年 03 月 25 日

1. 概述

1.1 案例介绍

本案例采用 FastCAE 的插件开发方式集成 Abaqus 模态分析功能，实现网格导入，仿真参数设置，求解参数设置，驱动 Abaqus 后台求解及结果可视化等流程。

本案例基于 FastCAE 提供的插件拓展机制，开发了 Abaqus 适配插件，实现了模态分析的全流程个性化定制，定义数据适配接口实现与 Abaqus 的数据传递，形成了基于 Abaqus 的专业定制模态分析软件。

本案例名称为 Abaqus 集成案例，该案例所有代码可以在网站 <https://pan.baidu.com/s/1iMdzMrrAzzy3e0zd0UvYVw>（提取码：51o7）处下载。

1.2 集成目标

- 使用 FastCAE 集成 Abaqus 实现模态分析功能的重新封装，固化分析流程，减少参数接口。通过本案例能够使用户快速了解使用 FastCAE2.0 进行插件开发方式的进行开发，为 Abaqus 软件集成与二次开发提供指导。

1.3 环境说明

- 开发环境
 - Windows 10

- VS2013 Community(VS 2013 其他版本也可以)
- Qt 5.4.2-msvc-opengl-x64
- FastCAE2.0
- ABAQUS 6.14-4

1.4 注意事项

- 建议开展本案例学习之前,要有一定 C++基础和 Abaqus 使用经验。
- 本次案例采用导入 Abaqus 中的 Inp 格式文件加载网格信息,并且在案例中尽量避免在使用 FastCAE 过程中代码存放位置及文件存放位置中包括中文路径。

2. 软件实现效果

本案例最终形成了基于 Abaqus 的专业定制模态分析软件,该软件数值计算由 Abaqus 程序实现,前后处理功能与流程通过 FastCAE 定制实现。前后处理的个性化流程定制以及前后处理与数值计算程序的数据交换通过 Abaqus 适配插件实现。软件界面如下图所示:

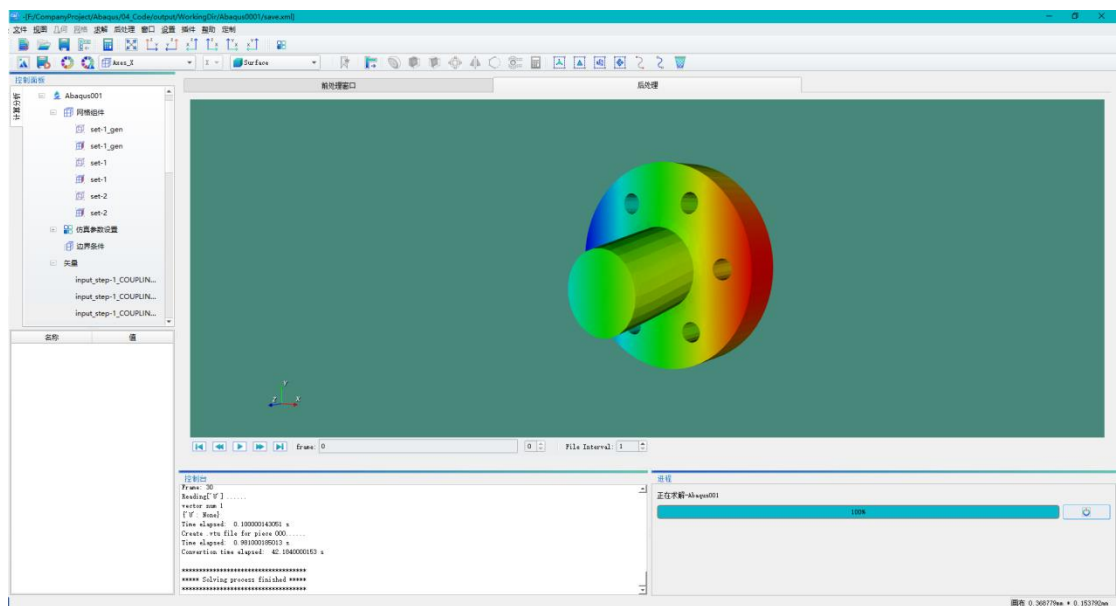


图 1 软件效果图

该专业定制的模式分析软件固化了分析流程，减少了大量的数据接口，在保证分析精度的前提下，降低了学习成本，提高了仿真分析的效率与可靠性。

3. 原理说明

本案例的核心集成原理，通过 FastCAE 平台软件定制界面，并生成 Abaqus 运行所需的脚本文件，后台驱动 Abaqus 软件进行科学计算并对计算结果进行展示。

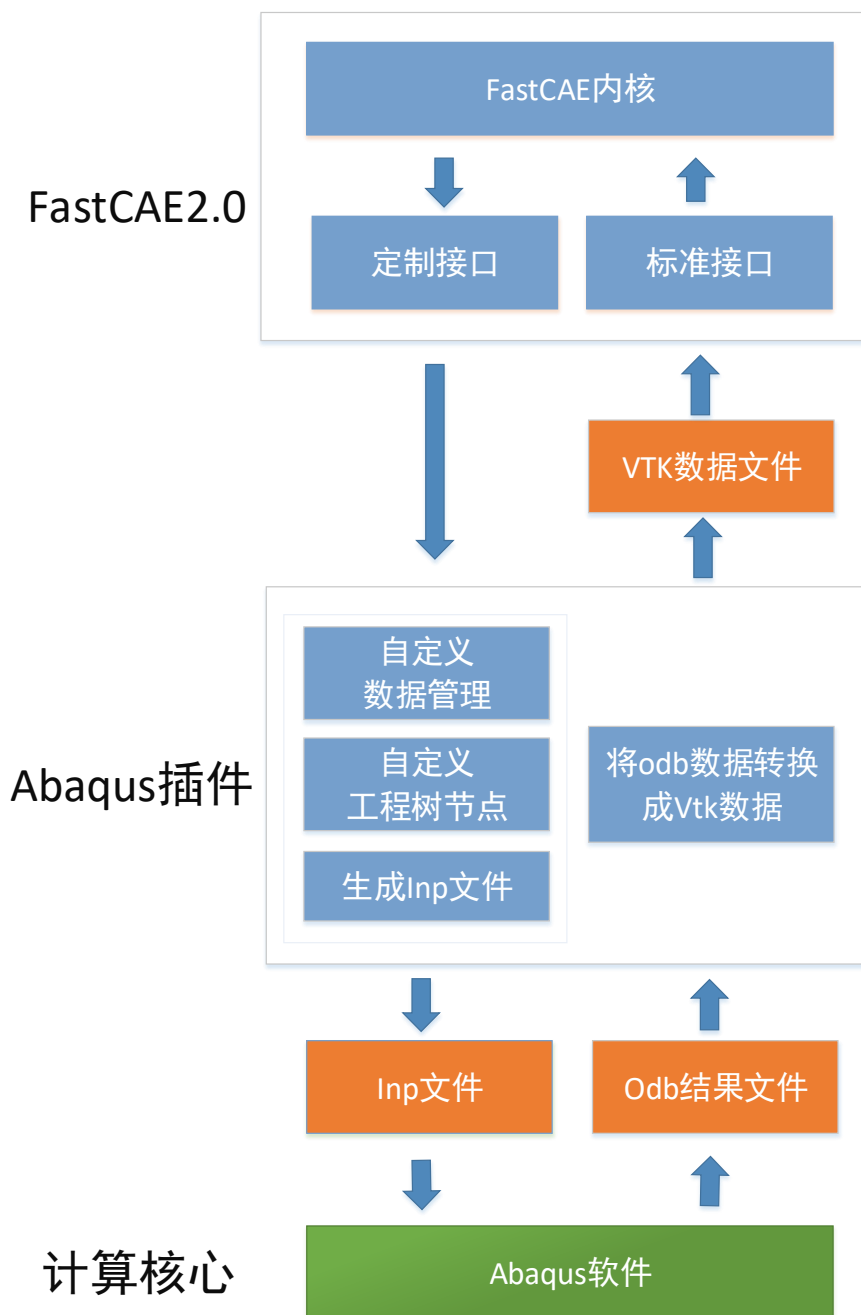


图 2 案例原理图

3.1 Abaqus 部分

- Abaqus 软件支持通过命令行的方式使用脚本进行启动运行。

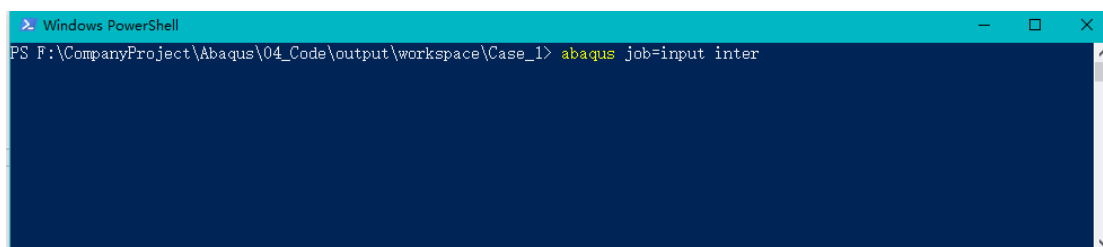


图 3Abaqus 命令

如上图，Abaqus 就会在后台加载 input.inp 文件进行运算。

Abaqus 命令行参数 job 表示提交分析作业。Inter 表示交互控制,显示启动计算情况。

3.2 FastCAE 部分

- 根据|原理说明-|Abaqus 部分内容，我们可以分析得出|FastCAE 部分需要调用 Abaqus 通过传参的形式让 Abaqus 在后台运行，即我们在|FastCAE 部分需要完成软件参数的设置、网络的导入及展示、边界条件设置、inp 文件内容的生成的、后处理结果展示的工作。

其中，FastCAE 部分及软件框架将提供网络的导入及展示。这部分功能不需要用户重新开发，只需要进行配置就能够使用。软件参数的设置、网络的导入及展示、边界条件设置、后处理结果展示功能以及 inp 文件生成功能将由插件完成。

4. 操作/开发步骤

首先我们需要进行插件开发，将当前 FastCAE2.0 版本不支持的功能通过插件的形式进行开发，通过平台加载的形式完成这部分功能的

支持。

通过插件定义一个自定义类型数据类，用来存储复杂数据。

通过插件定义重新定义工程树，满足流程需求。

将数据进行输出生成求解所需 Inp 格式文件。

由于我们开发的插件中具有将 odb 转换成 vtk 功能，这就需要在完成插件开发后，根据需求需要引入 “odb2vtk.py” 脚本文件。

然后我们需要按照我们软件的要求对软件的界面及功能进行定制。定制完成后，我们就可以开始我们所开发软件的使用旅程了！

4.1 Abaqus 插件开发

4.1.1 插件开发前准备

本次插件开发代码中，引用了许多 FastCAE 解决方案中的项目，需在 PluginAbaqus.pro 文件中声明引用其项目,如下图所示。

```
PluginAbaqus.pro
19 Release:UI_DIR          = ./release/qui
20 Release:OBJECTS_DIR     = ./release/obj
21 Release:LIBS += \
22 -L../../../../output/bin -lPluginManager \
23 -L../../../../output/bin -lModelData \
24 -L../../../../output/bin -lModuleBase \
25 -L../../../../output/bin -lDataProperty \
26 -L../../../../output/bin -lProjectTree \
27 -L../../../../output/bin -lMainWidgets \
28 -L../../../../output/bin -lProjectTreeExtend \
29 -L../../../../output/bin -lMeshData \
30 -L../../../../output/bin -lIO \
31 -L../../../../output/bin -lSelfDefObject \
32 -L../../../../output/bin -lPostWidgets \
33
34
35 Debug:CONFIG            += console
36 Debug:DESTDIR           = ../../../../output/bin_d/plugins
37 Debug:MOC_DIR           = ./debug/moc
38 Debug:RCC_DIR           = ./debug/rcc
39 Debug:UI_DIR            = ./debug/qui
40 Debug:OBJECTS_DIR       = ./debug/obj
41 Debug:LIBS += \
42 -L../../../../output/bin_d -lPluginManager \
43 -L../../../../output/bin_d -lModelData \
44 -L../../../../output/bin_d -lModuleBase \
45 -L../../../../output/bin_d -lDataProperty \
46 -L../../../../output/bin_d -lProjectTree \
47 -L../../../../output/bin_d -lMainWidgets \
48 -L../../../../output/bin_d -lProjectTreeExtend \
49 -L../../../../output/bin_d -lMeshData \
50 -L../../../../output/bin_d -lIO \
51 -L../../../../output/bin_d -lSelfDefObject \
52 -L../../../../output/bin_d -lPostWidgets \
53
```

图 4 Pro 项目文件

PluginAbaqus 项目文件路径存放在 FastCAE 工程下，保持相对路径一致，如下图。

BCBase	2020/3/11 16:50	文件夹
ConfigOptions	2020/3/11 16:50	文件夹
CurveAnalyse	2020/3/11 16:50	文件夹
DataProperty	2020/3/11 16:50	文件夹
DllsForDesigner	2019/11/28 11:13	文件夹
geometry	2020/3/11 16:50	文件夹
GeometryCommand	2020/3/13 9:29	文件夹
GeometryWidgets	2020/3/13 9:29	文件夹
Gmsh	2020/3/11 16:50	文件夹
GraphicsAnalyse	2020/3/11 16:50	文件夹
HeuDataSrcIO	2020/3/11 16:50	文件夹
IO	2020/3/11 16:53	文件夹
License	2020/3/11 16:50	文件夹
main	2020/3/11 16:50	文件夹
MainWidgets	2020/3/13 9:29	文件夹
mainWindow	2020/3/13 12:17	文件夹
Material	2020/3/11 16:50	文件夹
meshData	2020/3/11 16:50	文件夹
ModelData	2020/3/13 9:29	文件夹
moduleBase	2020/3/11 16:50	文件夹
ParaClassFactory	2020/3/11 16:50	文件夹
PluginAbaqus	2020/3/11 10:52	文件夹
PluginComplex	2019/11/28 11:49	文件夹
PluginCustomizer	2020/3/11 16:11	文件夹
PluginETS	2020/3/4 14:17	文件夹
PluginFluent	2020/3/4 16:53	文件夹
PluginHull	2020/3/11 16:11	文件夹

图 5 存放路径

4.1.2 插件通用部分代码修改

我们的插件继承自插件基类 `PluginBase`，因此基类的纯虚函数需要在我们的插件类中实现。这里我们把我们的插件类名字命名为 `PluginBase`。此时类所在的头文件 `PluginBase.h` 内容如下：

```
namespace Plugins
{
    class PLUGINABAQUSAPI PluginAbaqus : public PluginBase
    {
        Q_OBJECT
    }
}
```

```

        public:
            PluginAbaqus(GUI::MainWindow* m);
            ~PluginAbaqus();

            //加载插件
            bool install() override;
            //卸载插件
            bool uninstall() override;
            //翻译
            void reTranslate(QString lang) override;
        private:
            GUI::MainWindow* _mainwindow{};

    };
}
extern "C"
{
    //插件注册
    void PLUGINABAQUSAPI Register
        (GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps);
}

```

此时类所在的 C++ 文件 `PluginBase.cpp` 内容如下：

```

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
    Plugins::PluginBase* p = new Plugins::PluginAbaqus(m);
    ps->append(p);
}
namespace Plugins
{
    PluginAbaqus::PluginAbaqus(GUI::MainWindow* m)
    {
        _mainwindow = m;
    }
    PluginAbaqus::~PluginAbaqus()
    {
    }
    bool PluginAbaqus::install()
    {
        return true;
    }
    bool PluginAbaqus::uninstall()
    {
    }
}

```

```
        return true;
    }
}
```

在基类 `PluginBase` 中有一个 `Protected` 成员变量 `QString_describe`，它是用于描述当前插件信息的字符串变量。这里我们在 `PluginAbaqus` 类的构造函数中对其赋值。

```
_describe = " Abaqus plugin ";
```

4.1.3 插件需要完成函数选择

当前 FastCAE2.0 插件支持的输入输出扩展函数包括：

//注册写出的后缀与方法

```
static void RegisterInputFile(QString suffix, WRITEINPFILE fun);
```

//注册结果文件转换方法

```
static void RegisterOutputTransfer(QString name, TRANSFEROUTFILE fun);
```

//注册网格文件导入的方法

```
static void RegisterMeshImporter(QString suffix, IMPORTMESHFUN fun);
```

//注册网格导出的方法

```
static void RegisterMeshExporter(QString suffix, EXPORTMESHFUN fun);
```

需要注意的是 `RegisterInputFile` 函数注册完成后，当在求解器设置成我们所扩展的格式时，该函数会在求解器求解之前进行调用。而 `RegisterOutputTransfer` 函数注册完成后，会在完成求解器求解生成结果文件后调用。`RegisterMeshImporter` 和 `RegisterMeshExporter` 方法是为新格式的网格文件导入导出使用的函数。

根据我们的需求，本次案例中使用了导入 `inp` 文件读取网格数据和我们需要在求解器求解之前完成 `Inp` 文件生成。因此我们选择 `RegisterInputFile`、`RegisterMeshImporter` 方法作为我们拓展的方

法。因此我们在头文件中 C 函数部分添加：

```
//求解器文件写出
bool PLUGINABAQUSAPI WriteOut(QString path, ModelData::ModelDataBase* d);
```

在 CPP 文件中添加：

```
bool WriteOut(QString path, ModelData::ModelDataBase* d)
{
    qDebug() << d->getName();
    qDebug() << "writeTest";

    ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(d);

    QString strURL = model->getPath() + "/input.inp";
    InpWriteManager inp(model);
    inp.writeInit(strURL);

    strURL = model->getPath() + "/ODB2VTK+.cfg";
    ODB2VTKConfig cfg(model);
    cfg.writeConfig(strURL);

    QString sourcePath = QApplication::applicationDirPath() + "../Solve";
    QString sourceFile = sourcePath + "/ODB2VTK+.py";
    QString targetFile = model->getPath() + "/ODB2VTK+.py";
    QFile file(sourceFile);

    if (file.exists())
    {
        QFile::copy(sourceFile, targetFile);
    }

    QString strVtkUrl = model->getPath() + "/vtk";
    QDir dir;
    if (!dir.exists(strVtkUrl))
    {
        dir.mkpath(strVtkUrl);
    }

    return true;
}
```

WriteOut 函数中,我们要完成求解前所需要准备的输出数据,其

中包括了读取配置参数生成 inp 文件以及求解后生成的 odb 文件转换成 vtk 格式数据的配置文件, 都在求解前生成。主要实现是通过 `InpWriteManager` 读取 `AbaqusModel` 中的数据, 将其输出成 `input.inp` 文件, `ODB2VTKConfig` 读取 `AbaqusModel` 数据生成 python 脚本 `ODB2VTK+.py` 需要配置文件 `ODB2VTK+.cfg`。上述中 `InpWriteManager`、`AbaqusModel`、`ODB2VTKConfig` 会在本文后续中介绍。在 `WriteOut` 函数中指定了 `ODB2VTK+.py` 路径为 `applicationDirPath() + "/../Solve"`, 也就是 `output` 文件夹下的 `Solve` 中, 所以我们需要把 `ODB2VTK+.py` 拷贝到该路径下。

根据我们的需求, 我们需要自己维护一个复杂的数据类, 用于记录 `inp` 中使用的相关参数。因此我们选择 `registerType` 方法注册自定义数据类。因此我们在头文件中 `C` 函数部分添加:

```
//创建数据类
bool PLUGINABAQUSAPI CreateModel(int t, QPair<int, ModelData::ModelDataBase*>*
);
```

在 CPP 文件中添加:

```
bool CreateModel(int t, QPair<int, ModelData::ModelDataBase*>* p)
{
    if (t == ProjectTreeType::PluginDefType + 101)
    {
        auto m = new ModelData::AbaqusModel((ProjectTreeType)t);
        p->first = t;
        p->second = m;
        return true;
    }
    return false;
}
```

其中 `AbaqusModel` 是自定义数据类, 内容部分后续进行介绍。

接下来是重新定义工程树, 通过选择 `registerType` 方法注册自定

义工程树类。因此我们在头文件中 C 函数部分添加：

```
//创建树
bool PLUGINABAQUSAPI CreateTree
    (int, GUI::MainWindow* m, QPair<int, ProjectTree::ProjectTreeBase*>*)
;
```

在 CPP 文件中添加：

```
bool CreateTree(int tp, GUI::MainWindow* m, QPair<int, ProjectTree::ProjectTreeBase*>* t)
{
    if (tp == PluginDefType + 101)
    {
        t->first = tp;
        t->second = new ProjectTree::AbaqusTree(m);
        return true;
    }
    return false;
}
```

4.1.4 插件功能开发

插件 `PluginAbaqus` 最终头文件内容：

```
#ifndef PLUGINABAQUS_H
#define PLUGINABAQUS_H

#include "PluginAbaqusAPI.h"
#include "PluginManager\pluginBase.h"
#include <QObject>

namespace GUI
{
    class MainWindow;
}

namespace ProjectTree
{
    class ProjectTreeBase;
}

namespace ModelData
{
    class ModelDataBase;
}
```

```

namespace Plugins
{

    class PLUGINABAQUSAPI PluginAbaqus : public PluginBase
    {
        Q_OBJECT

    public:
        PluginAbaqus(GUI::MainWindow* m);
        ~PluginAbaqus();

        //加载插件
        bool install() override;
        //卸载插件
        bool uninstall() override;
        //翻译
        void reTranslate(QString lang) override;

    private:
        GUI::MainWindow* _mainwindow{};

    };
}

extern "C"
{
    //插件注册
    void PLUGINABAQUSAPI Register
        (GUI::MainWindow* m, QList<Plugins::PluginBase*>* plugs);

    //创建数据类
    bool PLUGINABAQUSAPI CreateModel(int t, QPair<int, ModelData::ModelDataBase*>* );

    //创建树
    bool PLUGINABAQUSAPI CreateTree

        (int, GUI::MainWindow* m, QPair<int, ProjectTree::ProjectTreeBase*>*)
        ;

    //求解器文件写出
    bool PLUGINABAQUSAPI WriteOut(QString path, ModelData::ModelDataBase*
        d);

    //网格导入方法

```

```

        bool PLUGINABAQUSAPI importMeshFun(QString filename);
    }

#endif // PLUGINABAQUS_H

```

插件 PluginAbaqus 最终 CPP 文件内容:

```

#include "pluginabacus.h"
#include "mainWindow/mainWindow.h"
#include "ModelData/modelDataFactory.h"
#include "ModelData/modelDataSingleton.h"
#include "ModelData/modelDataBase.h"
#include "MainWidgets/ProjectTreeFactory.h"
#include "abaqustree.h"
#include "PluginAbaqus\abacusmodel.h"
#include "IO/IOConfig.h"
#include "QDebug"
#include "inpwritermanager.h"
#include "QProcess"
#include "odb2vtkconfig.h"
#include "QDir"
#include "QCoreApplication"
#include <QApplication>
#include <QTranslator>
#include <assert.h>

Plugins::PluginAbaqus::PluginAbaqus(GUI::MainWindow* m)
{
    _mainwindow = m;
    _describe = "Abaqus plugin";
}

Plugins::PluginAbaqus::~~PluginAbaqus()
{
}

bool Plugins::PluginAbaqus::install()
{

```



```

        ModelData::ModelDataFactory::registerType(PluginDefType + 101, "Plugin
Abaqus", CreateModel);
        MainWindow::ProjectTreeFactory::registerType(PluginDefType + 101, Crea
teTree);
        IO::IOConfigure::RegisterInputFile("inp", WriteOut);

        return true;
    }

bool Plugins::PluginAbaqus::uninstall()
{
    ModelData::ModelDataFactory::removeType(PluginDefType + 101);
    MainWindow::ProjectTreeFactory::removeType(PluginDefType + 101);
    IO::IOConfigure::RemoveInputFile("inp");

    return true;
}

void Plugins::PluginAbaqus::reTranslate(QString lang)
{
    auto app = static_cast<QApplication*>(QCoreApplication::instance());
    app->removeTranslator(_translator);
    if (lang.toLower() == "chinese")
    {
        bool ok = _translator->load(":/translation/pluginabacus_zh");
        assert(ok);
        app->installTranslator(_translator);
    }
}

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* plugs)
{
    Plugins::PluginBase * p = new Plugins::PluginAbaqus(m);
    plugs->append(p);
}

bool CreateModel(int t, QPair<int, ModelData::ModelDataBase*>* p)
{
    if (t == ProjectTreeType::PluginDefType + 101)
    {

```

```

        auto m = new ModelData::AbaqusModel((ProjectTreeType)t);
        p->first = t;
        p->second = m;
        return true;
    }
    return false;
}

bool CreateTree(int tp, GUI::MainWindow* m, QPair<int, ProjectTree::ProjectTreeBase*>* t)
{
    if (tp == PluginDefType + 101)
    {
        t->first = tp;
        t->second = new ProjectTree::AbaqusTree(m);
        return true;
    }
    return false;
}

bool WriteOut(QString path, ModelData::ModelDataBase* d)
{
    qDebug() << d->getName();
    qDebug() << "writeTest";

    ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(d);

    QString strURL = model->getPath() + "/input.inp";
    InpWriteManager inp(model);
    inp.writeInit(strURL);

    strURL = model->getPath() + "/ODB2VTK+.cfg";
    ODB2VTKConfig cfg(model);
    cfg.writeConfig(strURL);

    QString sourcePath = QApplication::applicationDirPath() + "../Solve";

    QString sourceFile = sourcePath + "/ODB2VTK+.py";
    QString targetFile = model->getPath() + "/ODB2VTK+.py";
    QFile file(sourceFile);

```

```

        if (file.exists())
        {
            QFile::copy(sourceFile, targetFile);
        }

        QString strVtkUrl = model->getPath() + "/vtk";
        QDir dir;
        if (!dir.exists(strVtkUrl))
        {
            dir.mkpath(strVtkUrl);
        }

        return true;
    }
}

bool importMeshFun(QString)
{
    return true;
}

```

4.1.5 AbaqusModel 数据类开发

我们的自定义数据类继承自数据类扩展类 [ModelDataBaseExtend](#)，因此基类的纯虚函数需要在我们自定义类中实现。头文件 [AbaqusModel.h](#) 内容如下：

```

#ifndef ABAQUSMODEL_H
#define ABAQUSMODEL_H

#include "PluginAbaqusAPI.h"
#include "ModelData/modelDataBaseExtend.h"
#include "stepdata.h"
#include "bcdata.h"

struct MaterialStruct
{
    QString strMaterialName = "";
    QString strSetName = "";
    QString strDensity = "";
    QString strModulus = "";
    QString strRatio = "";
};

```

```

namespace ModelData
{

    class PLUGINABAQUSAPI AbaqusModel :public ModelDataBaseExtend
    {
    public:
        AbaqusModel(ProjectTreeType t);
        ~AbaqusModel();

        QHash<QString, StepData*>* getStepDataHash() const { return _stepDataHash; }

        QHash<QString, BCData*>* getBcDataHash() const { return _bcDataHash; }

        QList<QString>* getStepNameList() const { return _stepNameList; }

        MaterialStruct* getMaterialStruct() const { return _materialStruct; }

    private:
        QList<QString>* _stepNameList = nullptr;
        QHash<QString, StepData*>* _stepDataHash = nullptr;
        QHash<QString, BCData*>* _bcDataHash = nullptr;
        MaterialStruct* _materialStruct = nullptr;

    protected:

        QDomElement& writeToProjectFile(QDomDocument* doc, QDomElement* ele)
        override;

        void readDataFromProjectFile(QDomElement* e) override;

    private:
        void writeStepNameList(QDomDocument* doc, QDomElement* ele);

        void writeMaterialStruct(QDomDocument* doc, QDomElement* ele);
        void writeStepDataHash(QDomDocument* doc, QDomElement* ele);
        void writeBCDataHash(QDomDocument* doc, QDomElement* ele);

        void clearStepDataHash();
        void clearBCDataHash();
    }
}

```

```

        };
    }
#endif // ABAQUSMODEL_H

```

其中 **StepData** 是分析步类、**BCData** 边界参数类，内容会在后面介绍。因为继承数据类 **ModelDataBaseExtend**，需要实现基类的纯虚函数 **writeToProjectFile**、**readDataFromProjectFile** 实现保存项目参数和读取项目参数功能。

4.1.6 BCData 数据类开发

BCData 数据类记录边界相关数据参数类，主要记录边界名称、边界类型名称、分析步名称、组件名称以及边界类型。以及数据保存与读入功能的实现。

头文件内容：

```

#ifndef BCDATA_H
#define BCDATA_H

#include "PluginAbaqusAPI.h"
#include "QString"
#include <QDomElement>

namespace BCEnumType
{
    enum SymmetryType
    {
        XSYMM,
        YSYMM,
        ZSYMM,
        XASYMM,
        YASYMM,
        ZASYMM,
        PINNED,
    };
};

```

```

        ENCASTRE,
    };
}

struct DisplacementStruct
{
    bool  bU1  =false;
    bool  bU2  =false;
    bool  bU3  = false;
    bool  bUR1  = false;
    bool  bUR2  = false;
    bool  bUR3  = false;
};

class PLUGINABAQUSAPI BCData
{
public:
    BCData();
    ~BCData();

private:

    QString  strBCName;//边界名称
    QString  strBCType;//边界类型
    QString  strStepName;//分析步名称
    QString  strSetName;//组件名称

    DisplacementStruct  displacement;
    BCEnumType::SymmetryType  symmetryType;
};

#endif // BCDATA_H

```

4.1.7 StepData 数据类开发

StepData 数据类记录分析步相关数据参数类，主要有分析步名称、分析步类型、分析步(Frequency)基础数据以及分析步其他数据，以及数据保存与读入功能的实现。

头文件内容:

```
#ifndef STEPDATA_H
#define STEPDATA_H

#include "PluginAbaqusAPI.h"
#include "QString"
#include <QDomElement>

namespace StepEnumType
{
    enum NlgeomType
    {
        Off,
        On,
    };

    enum EigensolverType
    {
        Lanczos,
        Subspace,
        AMS,
    };

    enum EigenvaluesType
    {
        AllinFrequencyRange,
        Value,
    };

    enum BlockSizeType
    {
        Default,
        ValueSize,
    };

    enum NormalizeType
    {
        Displacement,
        Mass,
    };
}
```

```

struct StepBasicStruct
{

    //Basic
    QString value_description = "";

    StepEnumType::EigensolverType radio_Eigensolver = StepEnumType::EigensolverType::Lanczos;

    StepEnumType::EigenvaluesType radio_Eigenvalues = StepEnumType::EigenvaluesType::AllinFrequencyRange;
    QString value_Eigenvalues = " ";

    bool check_FrequencyShift = false;
    QString value_FrequencyShift = " ";

    bool check_MinFrequency = false;
    QString value_MinFrequency = " ";

    bool check_MaxFrequency = false;
    QString value_MaxFrequency = " ";

    bool check_IncludeAcoustic = false;

    StepEnumType::BlockSizeType radio_BlockSize = StepEnumType::BlockSizeType::Default;
    QString value_BlockSize = " ";

    StepEnumType::BlockSizeType radio_MaxBlockLanczos = StepEnumType::BlockSizeType::Default;
    QString value_MaxBlockLanczos = " ";

    bool check_UseSIMBased = false;
    bool check_IncludeResidual = false;

};

struct StepOtherStruct
{

```



```

        StepEnumType::NormalizeType radio_Normalize = StepEnumType::NormalizeT
ype::Displacement;

        bool check_Evaluate = false;
        double value_Evaluate = 0.0f;
    };

    class StepData
    {
    public:
        StepData();
        ~StepData();

    private:
        void writeStepBasic(QDomDocument* doc, QDomElement* ele);
        void writeStepOther(QDomDocument* doc, QDomElement* ele);

    private:
        StepBasicStruct stepBasic;//分析步基础数据
        StepOtherStruct stepOther;//分析步其他数据
        QString stepName;//分析步名称
        QString stepTypeName;//分析步类型名称

    };

#endif // STEPDATA_H

```

4.1.8 AbaqusTree 工程树类开发

AbaqusTree 工程树类是通过自定义对工程树进行编辑，主要是增加新的节点和左右鼠标键功能。AbaqusTree 继承 ProjectTreeWithBasicNode 工程树基类，通过实现基类中的虚函数，达到编辑工程树实现效果。

ProjectTreeWithBasicNode 头文件内容：

```

#ifndef ABAQUESTREE_H
#define ABAQUESTREE_H

#include "ProjectTree/ProjectTreeWithBasicNode.h"

```

```
#include "PluginAbaqusAPI.h"

namespace GUI
{
    class MainWindow;
}

class CustomPostWidget;

namespace ProjectTree
{
    enum TreeCustomItemType
    {
        none,
        StepCreate,
        MaterialWidget,
        BCCreate,
        CustomVector,
        PostWindow,
    };

    class PLUGINABAQUSAPI AbaqusTree : public ProjectTreeWithBasicNode
    {
        Q_OBJECT
    public:
        AbaqusTree(GUI::MainWindow* m);
        ~AbaqusTree();

    protected:

        virtual void initBasicNode(QTreeWidgetItem* root) override;

        //鼠标右键事件
        virtual void contextMenu(QMenu* menu) override;
        //鼠标左键单击事件
        virtual void singleClicked() override;
        //鼠标左键双击事件
        virtual void doubleClicked() override;
        //创建树
```

```

        virtual void createTree(QTreeWidgetItem* root, GUI::MainWindow* mainWindow) override;
        //更新网格子树
        virtual void updateMeshSubTree() override;

        virtual void reTranslate() override;

        virtual void updateTree() override;
    private:

        void disableItems();

        CustomPostWidget* _customPostWidget = nullptr;
        QTreeWidgetItem* _customVectortitem{};
    private slots:
        void refresh_slot();
        void close3DPostWindow(Post::PostWindowBase*);

};
#endif // ABAQUESTREE_H

```

通过实现 `initBasicNode` 函数进行对工程树节点追加以及隐藏的功能，在函数中添加了“StepCreate”、“SetMaterial”、“Vector”三个节点。通过 `disableItems` 函数将一些初始的树节点进行隐藏。也可以通过

实现如下：

```

void ProjectTree::AbaqusTree::initBasicNode(QTreeWidgetItem* root)
{
    ProjectTreeWithBasicNode::initBasicNode(root);

    disableItems();

    QTreeWidgetItem* stepCreateWidgetitem =
        new QTreeWidgetItem(_simulationSettingItem, TreeItemType::SelfDefineItem);
    stepCreateWidgetitem->setText(0, tr("StepCreate"));
}

```

```

        stepCreateWidgetitem->setData(0, Qt::UserRole + 2, StepCreate);

        QTreeWidgetItem* setMaterialtitem =

        new QTreeWidgetItem(_simulationSettingItem, TreeItemType::SelfDefineItem);

        setMaterialtitem->setText(0, tr("SetMaterial"));
        setMaterialtitem->setData(0, Qt::UserRole + 2, MaterialWidget);

        _customVectortitem = new QTreeWidgetItem(_root, TreeItemType::SelfDefineItem);
        _customVectortitem->setText(0, tr("Vector"));
        _customVectortitem->setData(0, Qt::UserRole + 2, CustomVector);
    }
}

void ProjectTree::AbaqusTree::disableItems()
{
    QStringList disableList;
    disableList << "Monitor"
                << "Vector"
                << "Boundary Type"
                << "Solver Setting"
                << "Post";

    setDisableItems(disableList);
}

```

通过实现 `singleClicked` 函数进行对工程树节点追加以及鼠标左键点击功能,通过点击实现“创建分析步”、“设置材料”、“边界条件”窗口等功能。

实现如下：

```

void ProjectTree::AbaqusTree::singleClicked()
{
    this->getCurrentItemData();

    qDebug() << _name << " single click";

    TreeItemType type = (TreeItemType)_currentItem->type();
    int index = _currentItem->data(0, Qt::UserRole + 2).toInt();
}

```

```

        if (StepCreate == index)
        {

            ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(_
data);

            StepCreateWidget stepCreate(model, _mainWindow);
            stepCreate.exec();

        }
        else if (MaterialWidget == index)
        {

            ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(_
data);

            SetMaterialWidget materialWidget(model, _mainWindow);
            materialWidget.exec();

        }
        else if (ProjectBoundaryCondation == type)
        {

            ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(_
data);

            BCCreateWidget bCCreate(model, _mainWindow);
            bCCreate.exec();

        }
        else if (PostWindow == index)
        {

            QString strName = _currentItem->text(0);

            if (_customPostWidget == nullptr)
            {

                ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(_
data);

                _customPostWidget = new CustomPostWidget(_mainWindow, model, model->
getID());

            }
            _customPostWidget->drawImage(strName);
            emit openPostWindowSig(_customPostWidget);

            emit showPostWindowInfo(_data->getID(), Post::PostWindowType::D3);
        }
        else

```

```

    {
        ProjectTreeWithBasicNode::singleClicked();
    }
}

```

通过实现 `contextMenu` 函数进行对工程树节点追加以及鼠标右键点击功能,通过右键菜单点击实现“刷新”结果数据功能。当计算完毕后,在算例路径下生成可视化数据,通过“刷新”检索加载数据到工程树“矢量”节点下。

实现如下:

```

void ProjectTree::AbaqusTree::contextMenu(QMenu* menu)
{
    qDebug() << _name << "    Right click";

    QAction* action = nullptr;
    TreeItemType type = (TreeItemType)_currentItem->type();
    int index = _currentItem->data(0, Qt::UserRole + 2).toInt();
    switch (type)
    {
        case ProjectBoundaryCondation:
            break;
        case SelfDefineItem:
            if (index == CustomVector)
            {
                action = menu->addAction(tr("Refresh"));

                connect(action, SIGNAL(triggered()), this, SLOT(refresh_slot()));
            }

            break;
        default:
            ProjectTreeWithBasicNode::contextMenu(menu);
    }
}

void ProjectTree::AbaqusTree::refresh_slot()
{
    ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(_
data);

```

```

        QString strURL = model->getPath() + "/vtk";

        QDir dir(strURL);

        if (!dir.exists())

            return ;

        QStringList nameFilters;

        nameFilters << "*.vtu";

        QStringList listFiles = dir.entryList(nameFilters, QDir::Files | QD
ir::Readable, QDir::Name);

        int count = listFiles.count();
        _customVectortitem->takeChildren();
        for (int i = 0; i < count; i++)
        {
            QTreeWidgetItem* postWindowitem =

            new QTreeWidgetItem(_customVectortitem, TreeItemType::ProjectPostCounte
rChild);

            postWindowitem->setText(0, listFiles.at(i));
            postWindowitem->setData(0, Qt::UserRole + 2, PostWindow);
        }
    }
}

```

4.1.9 StepCreateWidget 类开发

分析步创建窗口，用于创建某种类型的分析步骤，在“Initial”后追加新增的分析步骤，当存在多个分析步骤的时候，可以选择在某一分析步后插入新增项。程序类型中可根据分类选择分析的类型，每个分类中存在一个或多个分析类型，根据选择的分析类型不同，分析参数界面也会有所不同。本案例讲解，屏蔽了其余的选项，仅保留流程使用的必要选项。Ui 设计代码见

StepCreateWidget.ui 文件。

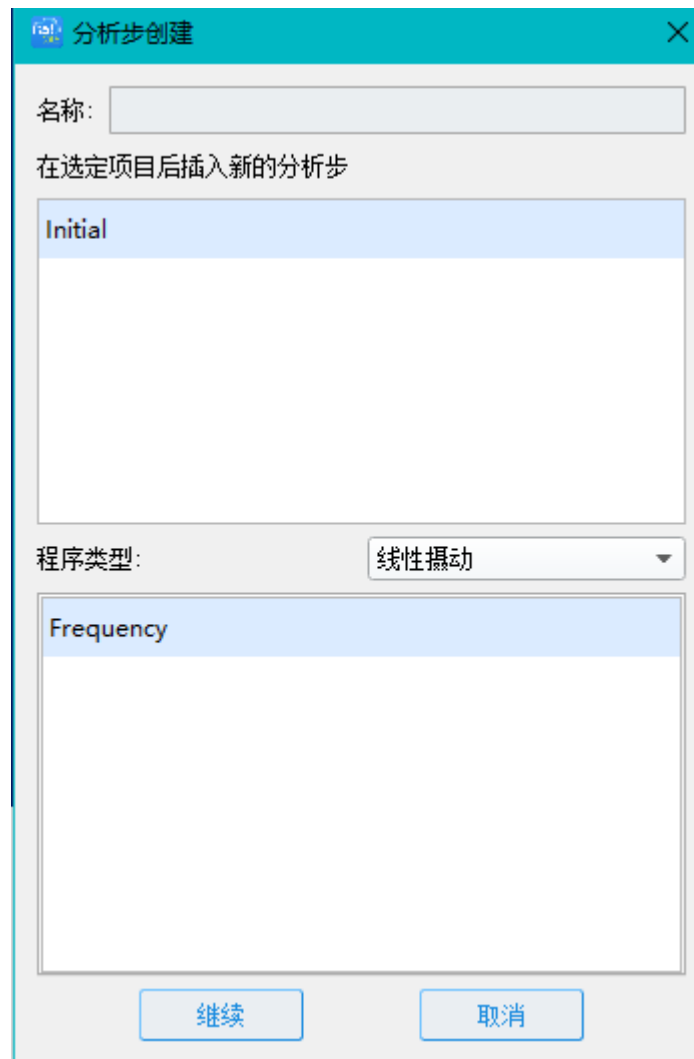


图 6 分析步创建

StepCreateWidget 头文件内容：

```
#ifndef STEPCREATEWIDGET_H
#define STEPCREATEWIDGET_H

#include "PluginAbaqusAPI.h"
#include "SelfDefObject/QFDialog.h"

namespace Ui {class StepCreateWidget;};

namespace ModelData
{
    class AbaqusModel;
}
```



```

namespace GUI
{
    class MainWindow;
}

class QListWidgetItem;

class PLUGINABAQUSAPI StepCreateWidget : public QDialog
{
    Q_OBJECT

public:
    StepCreateWidget(ModelData::AbaqusModel* model, GUI::MainWindow* m);
    ~StepCreateWidget();

protected:
    virtual void showEvent(QShowEvent *)override;

private:
    Ui::StepCreateWidget *ui;
    ModelData::AbaqusModel * _abaqusModel = nullptr;
    GUI::MainWindow* _mainWindow{};

    void init();
    void initStepList(); //初始化分析步列表
    void initProcedure(); //初始化下拉框列表
    void initGeneral(); //初始化通用类型列表
    void initLinearPerturbation(); //初始化线性摄动列表
    void initConnect(); //初始化信号槽

    QString checkDataValue();

private slots:

    void comboBoxTypeChange_slot(const QString&);
    void bcListDoubleClick_slot(QListWidgetItem *item);

    void on_pushButton_Continue_clicked();
    void on_pushButton_Cancel_clicked();
};

```

```
#endif // STEP_CREATE_WIDGET_H
```

其中在 `comboBoxTypeChange_slot` 函数中根据 UI 中默认显示“线性摄动”分析分类，在“程序类型”列表中显示“线性摄动”分类下的分析类型，在 `initLinearPerturbation` 函数中，实现了显示“Frequency”分析类型。

具体实现如下：

```
QString StepCreateWidget::checkDataValue()
{
    QString strError;
    QString strStepName = ui->lineEdit_StepName->text().trimmed();

    if (strStepName == "")
    {
        strError.append(tr("The Step Name is Empty. "));
    }
    return strError;
}

void StepCreateWidget::comboBoxTypeChange_slot(const QString& strText)
{
    if (strText.contains(tr("General")))
    {
        ui->tabWidget->setCurrentIndex(0);
    }
    else if (strText.contains(tr("Linear perturbation")))
    {
        ui->tabWidget->setCurrentIndex(1);
    }
}
```

4.1.10 Stepeditwidget 界面类开发

在分析步编辑界面中，因不同分析类型，分析编辑界面参数差异较大，本次案例仅展示“Frequency”类型的参数界面。

StepeditwidgetUI 界面显示如下：

StepEditWidget

Name: step-1

Type: Frequency

Basic Other

Description:

Nlgeom: Off

Eigsolver: ☒ Lanczos ☐ Subspace ☐ AMS

Number of eigenvalues requested: ☒ All in frequency range
☐ Value:

☐ Frequency shift (cycles/time)**2:

☐ Minimum frequency of interest(cycles/time):

☐ Maximum frequency of interest(cycles/time):

☐ Include acoustic-structural coupling where applicable

Block size: ☒ Default ☐ Value:

Maximum number of block Lanczos steps: ☒ Default ☐ Value:

☐ Use SIM-based linear dynamics procedures

☐ Include residual modes

OK Cancel

图 7 StepEditWidget Basic 界面

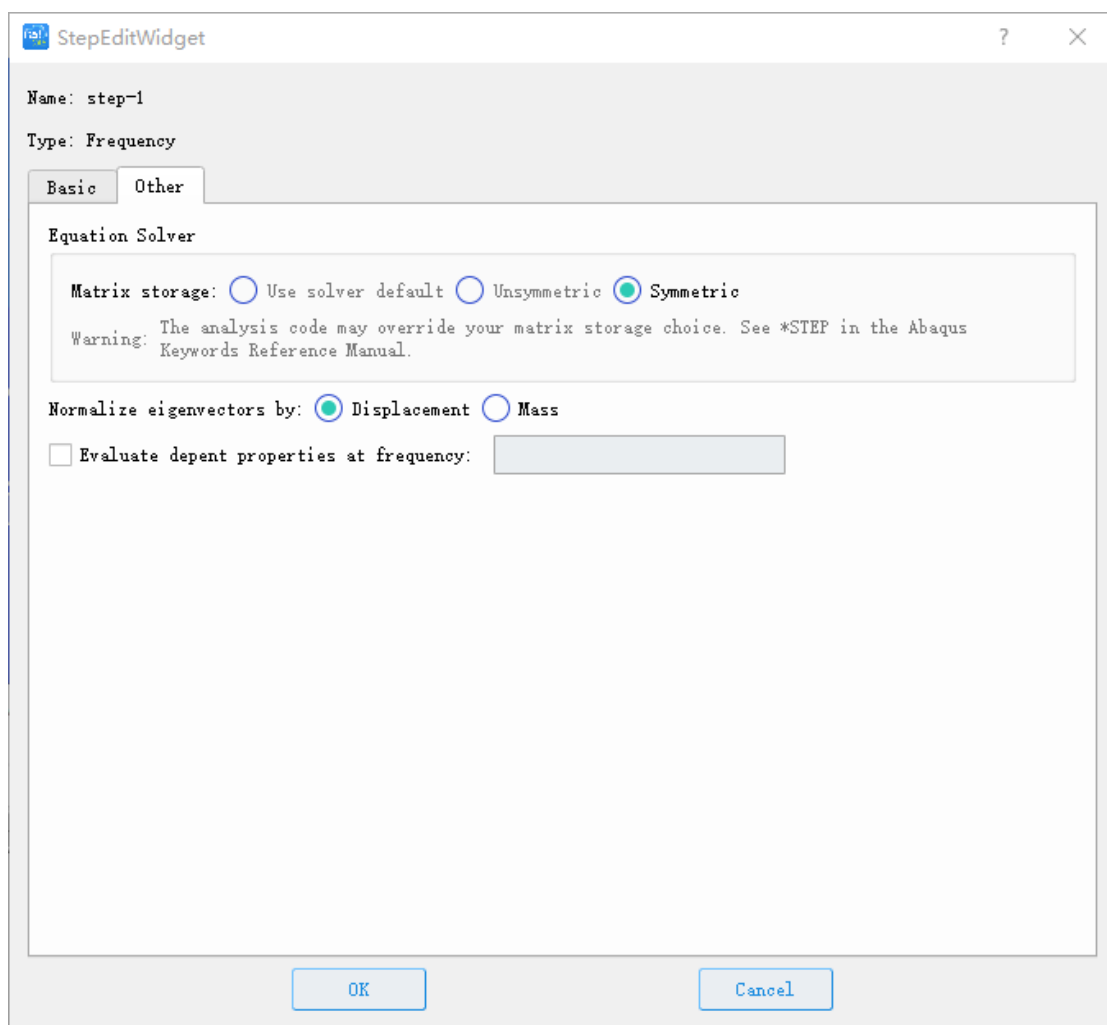


图 8StepEditWidget Other 界面

`Stepeditwidget` 界面类的主要功能是对分析参数的设置，“Frequency”分析类型中包含的参数如上图所示，本次案例使用到的参数有“Description:”和“Number of eigenvalues requested:Value”，其他参数均使用默认值。设定参数完毕后，会将分析步及参数存储到数据类 `AbaqusModel` 中。

`Stepeditwidget` 界面类头文件内容：

```
#ifndef STEPEDITWIDGET_H
#define STEPEDITWIDGET_H

#include <QDialog>
namespace Ui {class StepEditWidget;};
```

```

namespace ModelData
{
    class AbaqusModel;
}

class StepData;

class StepEditWidget : public QDialog
{
    Q_OBJECT

public:
    StepEditWidget(ModelData::AbaqusModel* model, QDialog *parent = 0);
    ~StepEditWidget();

    void SetStepNameAndType(QString strName, QString strType, int index);

    void SetStepNameAndType(QString strName, int index);

private:
    Ui::StepEditWidget *ui;
    ModelData::AbaqusModel* _abaqusModel = nullptr;
    StepData* _stepData = nullptr;
    int _StepIndex = 0;

    void initStepNameAndType(QString strName); //初始化分析步骤名称类型
    void initBasic(); //初始化基础界面
    void initOther(); //初始化其他界面
    void init();

private slots:
    void on_pushButton_OK_clicked(); //保存数据
    void on_pushButton_Cancel_clicked(); //取消
    void radioButtonClicked_slot(int); //单选信号槽
    void checkBoxClicked_slot(); //复选信号槽

};

#endif // STEPEDITWIDGET_H

```

4.1.11 SetMaterialWidget 界面类开发

SetMaterialWidget 界面类是维护材料类型的界面,本案例中使用

到的参数有“Mass Density”、“Young's Modulus”、“Poisson's Ratio”以及材料名称和组件。其作用是给某组件设定材料属性。设定参数完毕后，会将材料参数存储到数据类 `AbaqusModel` 中。

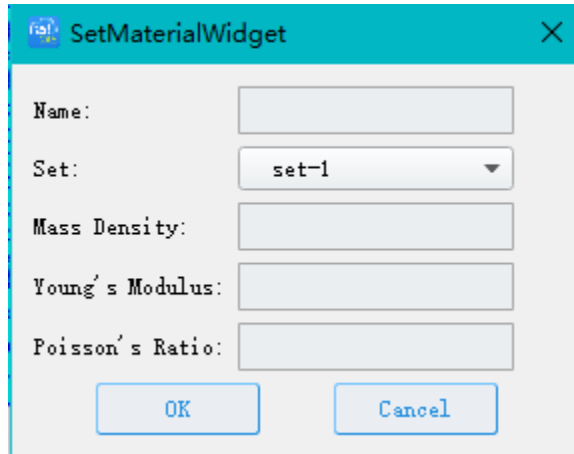


图 9 SetMaterialWidget 界面

SetMaterialWidget 界面类头文件内容：

```
#ifndef SETMATERIALWIDGET_H
#define SETMATERIALWIDGET_H

#include "PluginAbaqusAPI.h"
#include "SelfDefObject/QFDialog.h"

namespace Ui {class SetMaterialWidget;};

namespace ModelData
{
    class AbaqusModel;
}

namespace GUI
{
    class MainWindow;
}

class PLUGINABAQUSAPI SetMaterialWidget : public QFDialog
{
    Q_OBJECT

public:
    SetMaterialWidget(ModelData::AbaqusModel* model, GUI::MainWindow* m);
```

```

~SetMaterialWidget();

protected:
    virtual void showEvent(QShowEvent *)override;

private:
    Ui::SetMaterialWidget *ui;
    ModelData::AbaqusModel* _abaqusModel = nullptr;
    GUI::MainWindow* _mainWindow{};

    void initList();//初始化下来列表

    QString checkData();

    void initData();//初始化数据

private slots:

    void on_pushButton_OK_clicked();
    void on_pushButton_Cancel_clicked();

};

#endif // SETMATERIALWIDGET_H

```

4.1.12 BCCreateWidget 界面类开发

BCCreateWidget 界面类是提供选择设定边界条件类型界面类，可以设定在某分析步骤中某组件的边界类型，类型分类有力学、流体等，本案例仅演示案例使用到的力学中用到的两个类型：“Symmetry/Antisymmetry/Encastre”、“Displacement/Rotation”。

用户通过设定“分析步”、选择“组件”以及边界类型为模型设定边界条件，根据边界条件类型的不同，显示参数界面。右侧列表中显示已设定的边界条件。

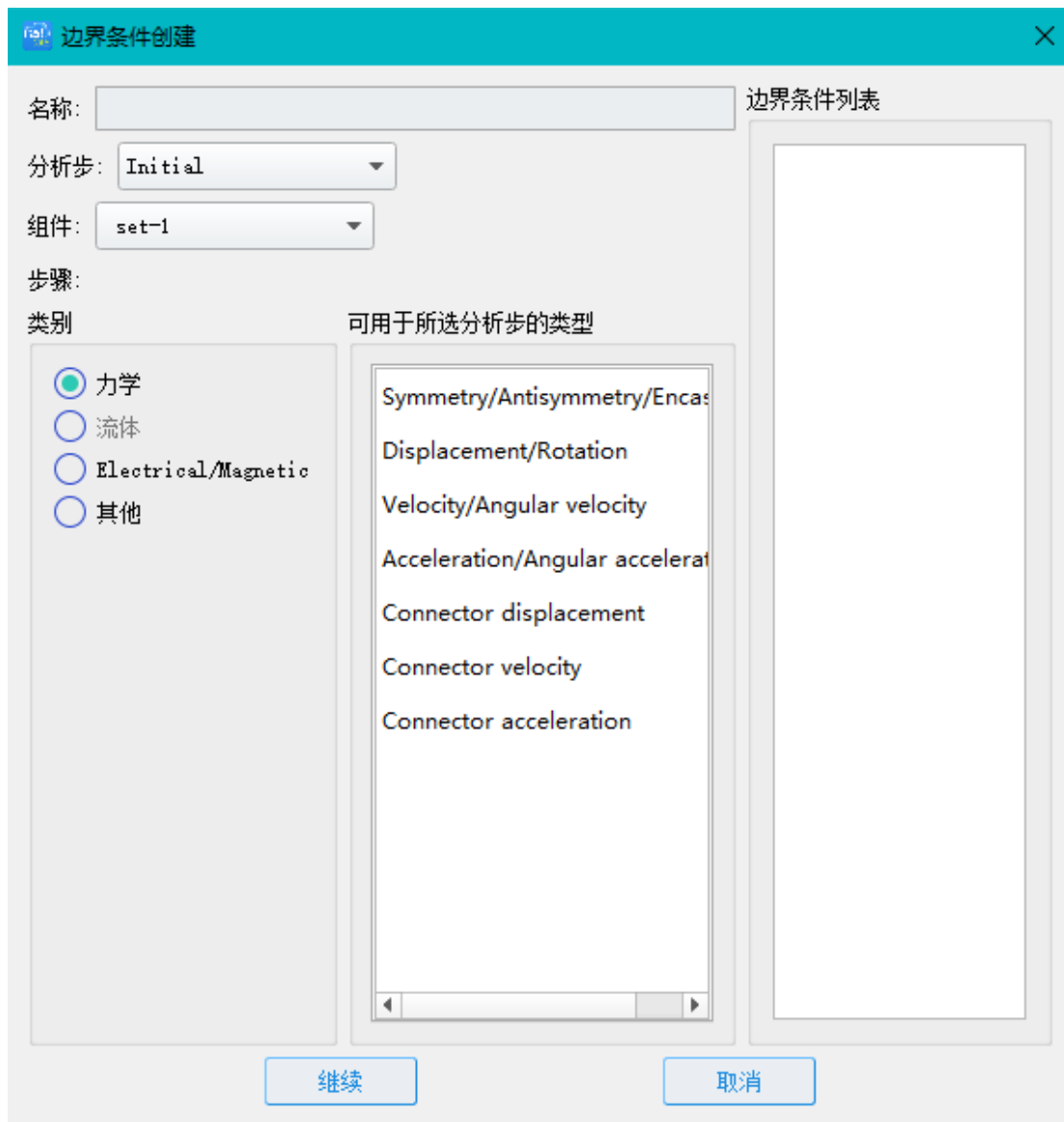


图 10 边界条件界面

BCCreateWidget 界面类头文件内容：

```
#ifndef BCCREATEWIDGET_H
#define BCCREATEWIDGET_H

#include "PluginAbaqusAPI.h"
#include "SelfDefObject/QFDialog.h"

namespace Ui {class BCCreateWidget;};

namespace ModelData
{
    class AbaqusModel;
}
```



```

class QListWidgetItem;

namespace GUI
{
    class MainWindow;
}

class PLUGINABAQUSAPI BCCreateWidget : public QFileDialog
{
    Q_OBJECT

public:
    BCCreateWidget(ModelData::AbaqusModel* model, GUI::MainWindow* m);
    ~BCCreateWidget();

protected:
    virtual void showEvent(QShowEvent *) override;

private:
    Ui::BCCreateWidget *ui;
    ModelData::AbaqusModel* _abaqusModel = nullptr;
    GUI::MainWindow* _mainWindow{};

    void init(); //初始化
    void initConnect(); //初始化信号槽
    void initRadio(); //初始化单选按钮
    void initMechanical(int stepType); //初始化力学
    void initElectrical(); //初始化电气
    void initOther(int stepType); //初始化其他
    void initStep(); //初始化分析步
    void initSet(); //初始化组件
    void initList(); //初始化列表
    QString checkData(); //数据校验
    void openEditWidget(QString strBCName, QString strStepName, QString
strSetName, QString strBCTypeName); //打开编辑界面

private slots:

    void radioCheck_slot(bool b);
    void bcListDoubleClick_slot(QListWidgetItem *item);

    void on_pushButton_Continue_clicked();

```

```

void on_pushButton_Cancel_clicked();

};

#endif // BCCREATEWIDGET_H

```

4.1.13 BCEditWidget 界面类开发

BCEditWidget 界面类，根据边界类型的不同，显示不同参数界面，编辑完毕后，将边界参数存储到自定义数据类 `AbaqusModel` 中。

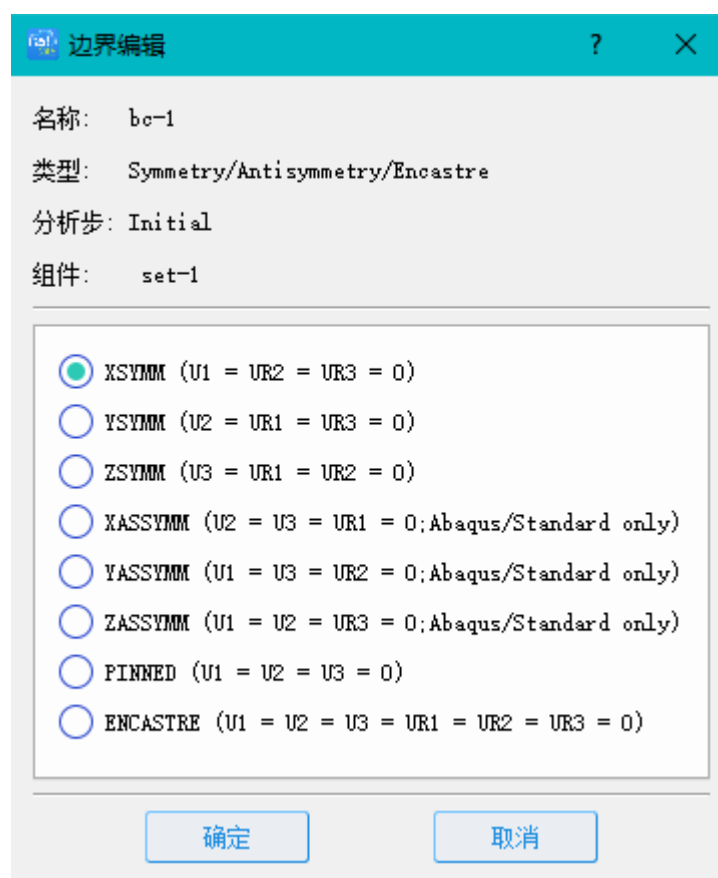


图 11 边界编辑界面

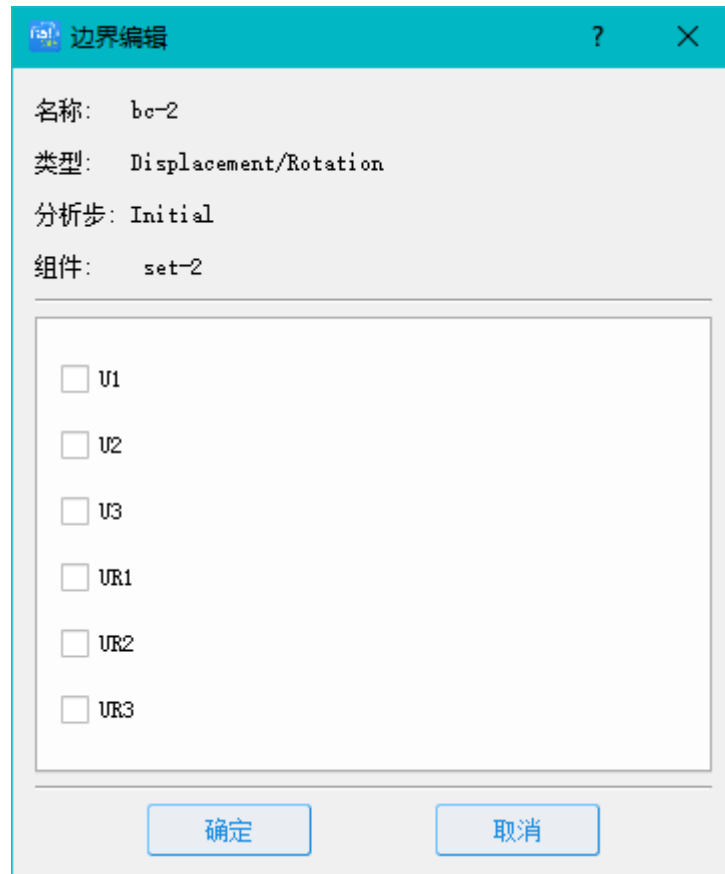


图 12 边界编辑界面

BCEditWidget 界面类头文件内容:

```
#ifndef BCEDITWIDGET_H
#define BCEDITWIDGET_H

#include <QDialog>

namespace Ui {class BCEditWidget;};

namespace ModelData
{
    class AbaqusModel;
}

class QButtonGroup;

class BCEditWidget : public QDialog
{
    Q_OBJECT

public:
```

```

        BEditWidget(ModelData::AbaqusModel* model, QDialog *parent = 0);
        ~BEditWidget();

        void initDataValue(QString strBCName, QString strStepName, QString str
SetName, QString strBCTypeName);

private:
    Ui::BEditWidget *ui;
    ModelData::AbaqusModel* _abacusModel = nullptr;
    QButtonGroup* _group = nullptr;

    void init(); //初始化
    void initRadioGroup(); //初始化单元组
    void initModelData(); //初始化数据
private slots:
    void on_pushButton_OK_clicked();
    void on_pushButton_Cancel_clicked();
};

#endif // BCEDITWIDGET_H

```

4.1.14 InpWriteManager 类开发

InpWriteManager 类是提供了将自定义数据类 AbaqusModel 中的数据转换为 Abaqus 的 Inp 格式文件功能，目前仅实现本案例所需数据的转换功能，并不能够支持任意数据转换功能。

InpWriteManager 类头文件内容：

```

#ifndef INPWRITEMANAGER_H
#define INPWRITEMANAGER_H

#include "QFile"
#include "QTextStream"
#include "PluginAbaqusAPI.h"

namespace ModelData
{
    class AbaqusModel;
}

```

```

class BCDData;

class PLUGINABAQUSAPI InpWriteManager
{
public:
    InpWriteManager(ModelData::AbaqusModel* model);
    ~InpWriteManager();
    void writeInit(QString fileURL); //根据 Abaqus 格式生成 inp 文件
private:
    ModelData::AbaqusModel* _abaqusModel = nullptr;

    void writeHeading(QTextStream* out);
    void writePart(QTextStream* out);
    void writeNode(QTextStream* out);
    void writeElement(QTextStream* out);
    void writeNset(QTextStream* out, QString strSetName, bool isInstance);
    void writeSection(QTextStream* out);
    void writeAssembly(QTextStream* out);
    void writeMaterial(QTextStream* out);
    void writeStep(QTextStream* out);
    void writeBC(QTextStream* out, QString strStepName);
    void writeOutput(QTextStream* out);

    void writeDisplacement(QTextStream* out, BCDData* bcData);
    void writeSymmetry(QTextStream* out, BCDData* bcData);

};

#endif // INPWRITEMANAGER_H

```

InpWriteManager 类中 writeNode 函数展示了如何获取网格组件中的网格节点数据并进行输出。

```

void InpWriteManager::writeNode(QTextStream* out)
{
    MeshData::MeshData* md = MeshData::MeshData::getInstance();
    if (md->getKernelCount() == 0)
    {
        return;
    }
    MeshData::MeshKernel* kernel = md->getKernelAt(0);
}

```

```

if (kernal != nullptr)
{
    *out << "*Node" << "\r\n";
    int count = kernal->getPointCount();
    for (int i = 0; i < count; i++)
    {
        double* coor = kernal->getPointAt(i);

        int pointIndex = i;

        QString strCoor0 = QString::number(coor[0], 'f', 11);

        QString strCoor1 = QString::number(coor[1], 'f', 11);

        QString strCoor2 = QString::number(coor[2], 'f', 11);

        QString s = QString("%1,%2,%3,%4")

        .arg(pointIndex + 1, 7, 10, QLatin1Char(' '))
          .arg(strCoor0, 17, QLatin1Char(' '))
          .arg(strCoor1, 17, QLatin1Char(' '))
          .arg(strCoor2, 17, QLatin1Char(' '));

        *out << s << "\r\n";
    }
}
}

```

InpWriteManager 类中其他函数是按照 Abaqus 格式进行模拟输出生成 Inp 文件。

4.1.15 ODB2VTKConfig 类开发

ODB2VTK+.py 是 python 通过 Abaqus 命令解析 odb 文件将内部数据按照 vtk 格式进行转译,其中主要是读取了网格、节点数据以及位移数据等。ODB2VTKConfig 类是根据算例路径及算例参数生成的配置文件,其中包含了 odb 文件名称、路径、以及读取的数据类型等命令配置。头文件内容:

```

#ifndef ODB2VTKCONFIG_H

```

```

#define ODB2VTKCONFIG_H

#include "QFile"
#include "QTextStream"
#include "PluginAbaqusAPI.h"

namespace ModelData
{
    class AbaqusModel;
}

class PLUGINABAQUSAPI ODB2VTKConfig
{
public:
    ODB2VTKConfig(ModelData::AbaqusModel* model);
    ~ODB2VTKConfig();

    void writeConfig(QString fileName);

private:
    ModelData::AbaqusModel* _abaqusModel = nullptr;

    void initPath(QTextStream* out);
    void initMesh(QTextStream* out);
    void initPartition(QTextStream* out);
    void initSetting(QTextStream* out);
    void initOutput(QTextStream* out);
    void initMultiprocessing(QTextStream* out);
    void initScale(QTextStream* out);

};

#endif // ODB2VTKCONFIG_H

```

4.1.16 CustomPostWidget 类开发

CustomPostWidget 类继承 PostWindowBase 基类，通过 drawImage 函数将要显示的 vtU 文件路径传递给后处理窗口进行显示。

头文件内容:

```
#ifndef CUSTOMPOSTWIDGET_H
#define CUSTOMPOSTWIDGET_H

#include <QWidget>
#include "PostWidgets/PostWindowBase.h"
#include "PluginAbaqusAPI.h"

namespace Ui {class CustomPostWidget;};

namespace Post
{
    class Post3DWindow;
}

namespace ModelData
{
    class AbaqusModel;
}

class PLUGINABAQUSAPI CustomPostWidget : public Post::PostWindowBase
{
    Q_OBJECT

public:
    CustomPostWidget

    (GUI::MainWindow* mainwindow, ModelData::AbaqusModel* model, int projectid);
    ~CustomPostWidget();

    void drawImage(QString fileName);

private:
    Ui::CustomPostWidget *ui;
    Post::Post3DWindow * _post3DWindow = nullptr;
    ModelData::AbaqusModel* _model = nullptr;
    int _projectID = 0;

    void initPost3D();

};

#endif // CUSTOMPOSTWIDGET_H
```


4.2 软件使用

1. 我们将开始运行我们开发的程序进行求解工作。鼠标左键点击“控制面板” - “计算分析” - “算例”节点，点击鼠标右键软件将弹出“算例”节点的右键菜单。



图 13 控制面板

2. 点击“算例”节点弹出的右键菜单中的“创建算例”菜单项，软件会弹出创建算例对话框。这里我们使用默认的名称，直接点击“确认”按钮完成算例的创建。如果有需要用户可以输入其他的算例名称。

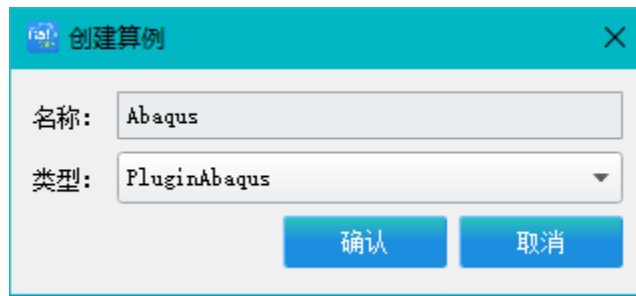


图 14 创建算例

3. 选择菜单“文件”下的“导入网格”，选择 inp 文件进行导入。

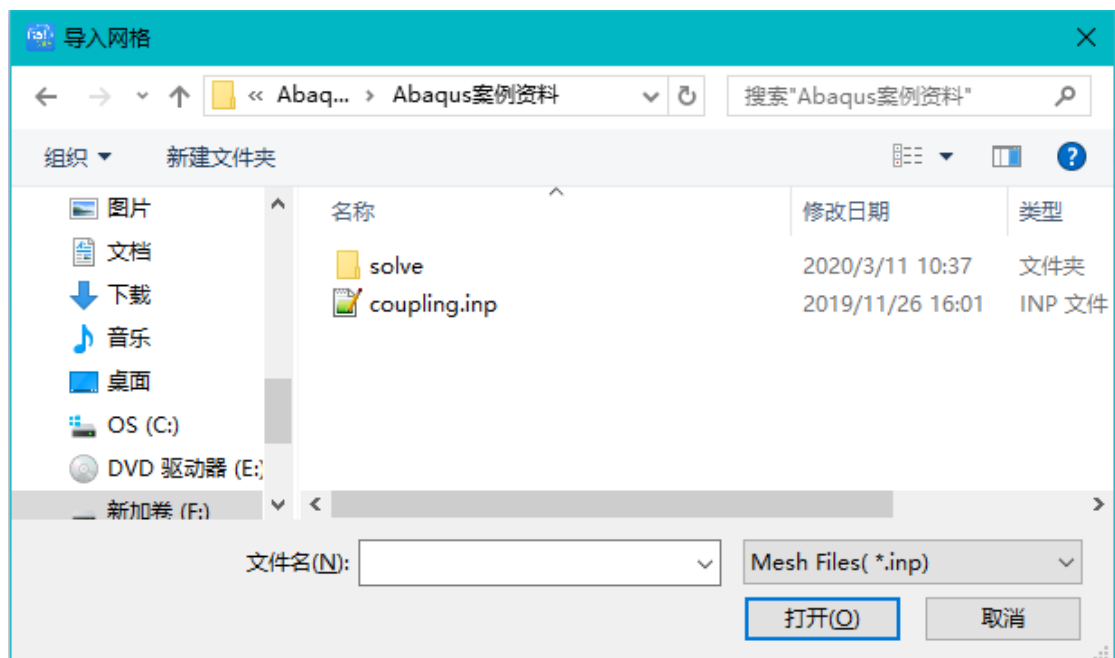


图 15 导入网格

4. 导入完毕后主界面显示如下图所示。

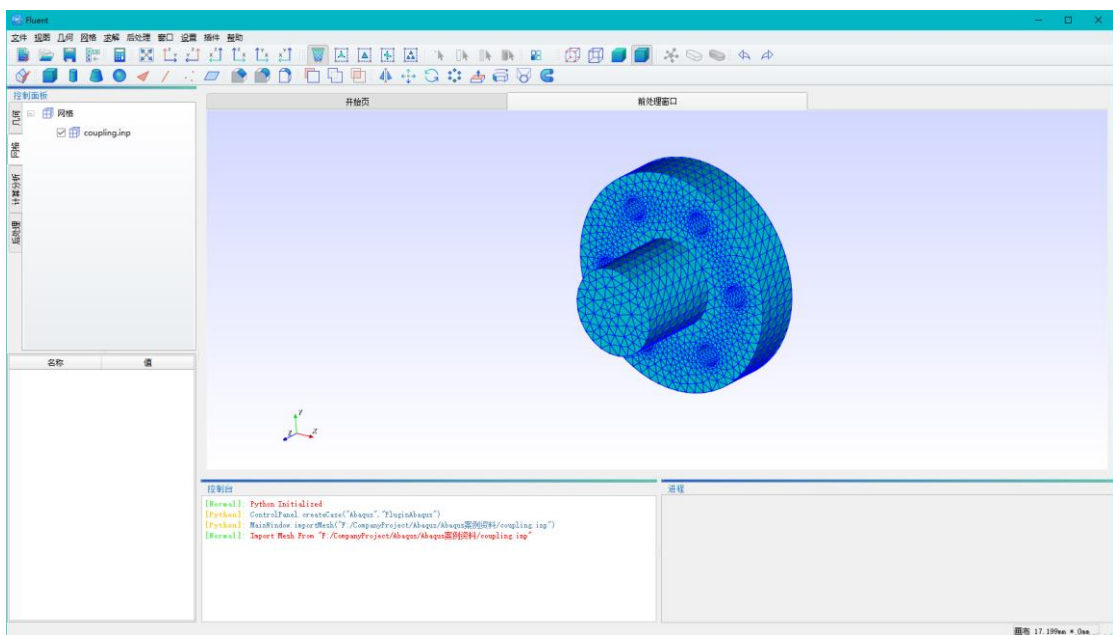


图 16 主界面

5. 此时算例下的树形结构如下图所示。点击“算例”下的“仿真参数设置”节点，展开后，可以看到分析步创建和设置材料节点。



图 17 控制面板

6. 点击分析步创建，显示分析步创建。名称填写“step-1”，选择分析

步“Initial”和选择“线性摄动”下的“Frequency”点击“继续”。

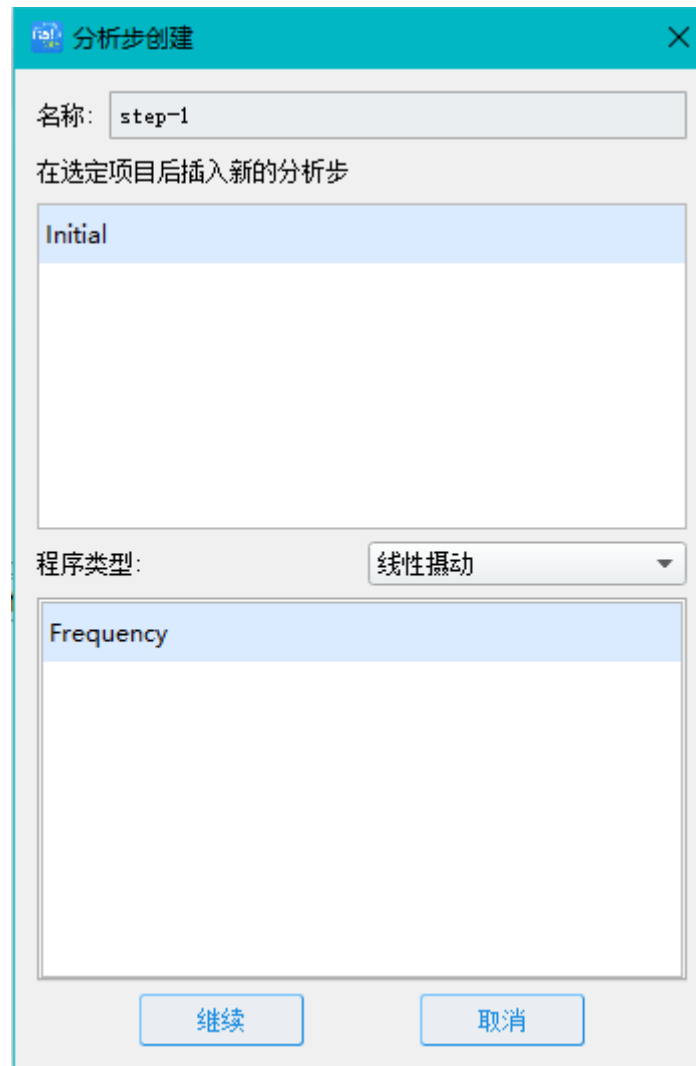


图 18 分析步创建

7. 点击分析步编辑界面中，在“描述”中填写：**Model analysis of a coupling**,在请求的特征值个数中选择“数值”并填写：30，其他参数均为默认值，点击“确定”。

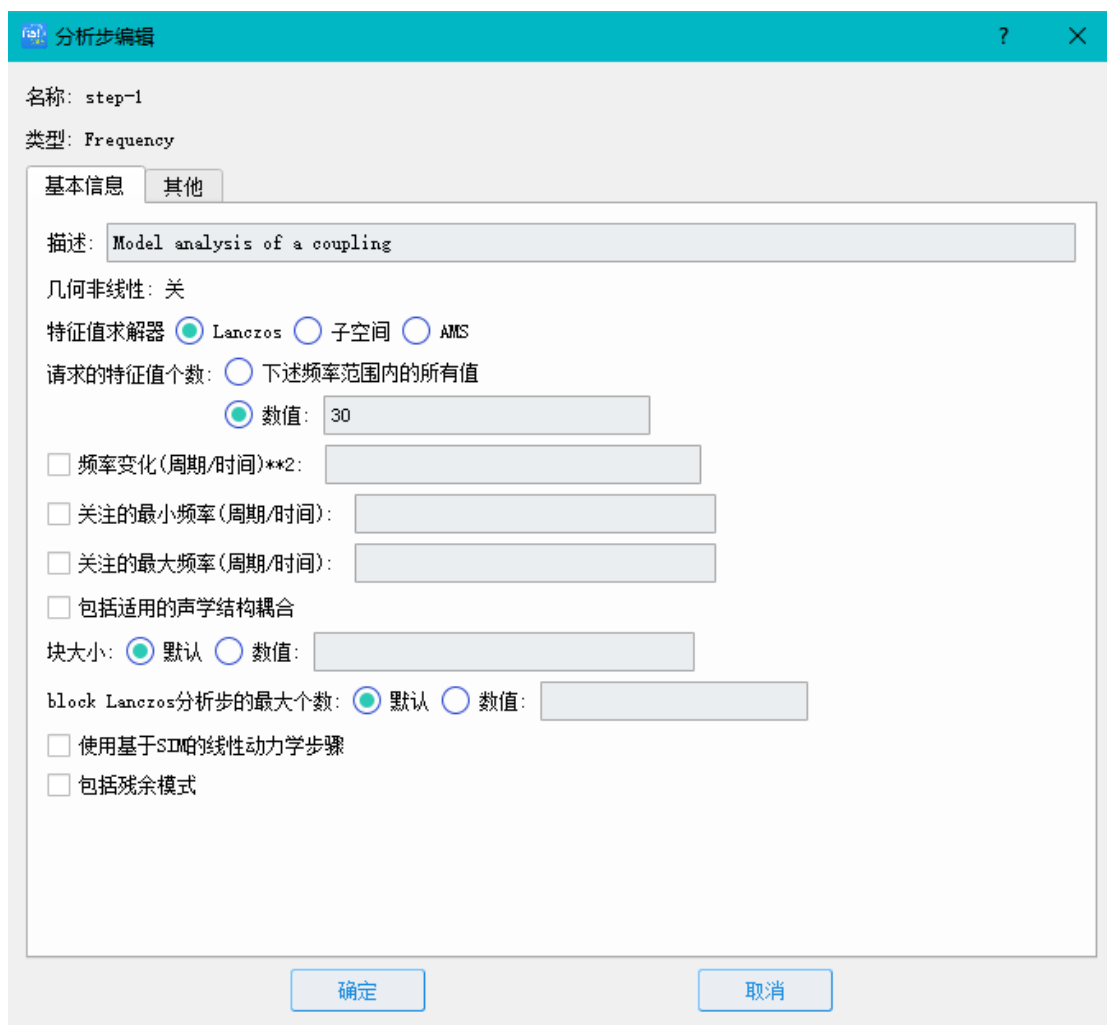


图 19 分析步编辑

8. 在工程树中选择“网格组件”右键点击“导入”，如下图。

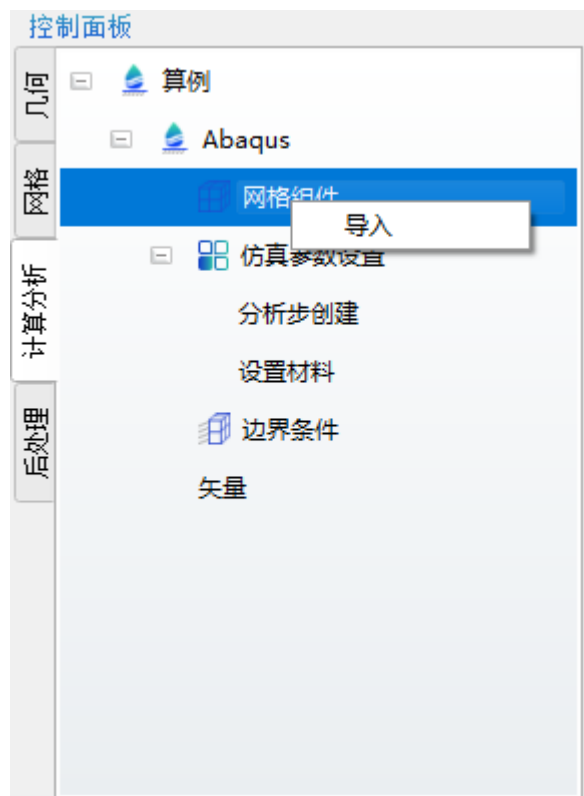


图 20 控制面板



图 21 导入界面

9. 将“可选”中的所有组件选中点击“>>”将其移动到“已选”列表中，
点击“确定”。



图 22 导入界面

10. 点击工程树中“设置材料”，分别设置以下参数数值，如图：



图 23 设置材料界面

11. 点击工程树中“边界条件”，打开边界条件创建界面，在界面中填写名称“bc-1”，选择分析步“Initial”，组件选择“set-1”，类别选择“力学”下的“Displacement/Rotation”，点击“继续”。

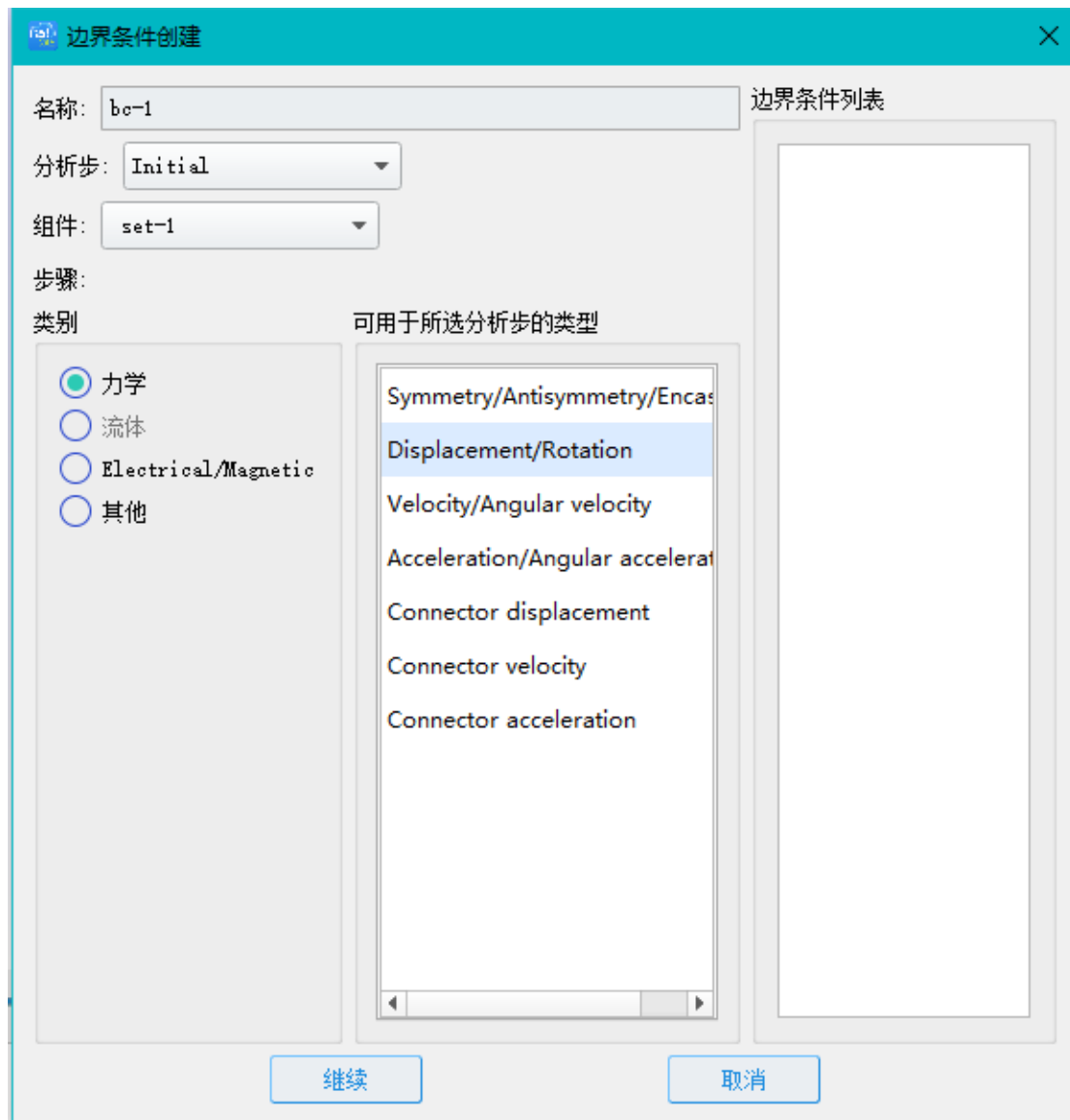


图 24 边界条件创建

12. 在边界编辑界面中，选中“U1”、“U2”、“U3”、“UR1”、“UR1”，
点击“确定”，如下图。

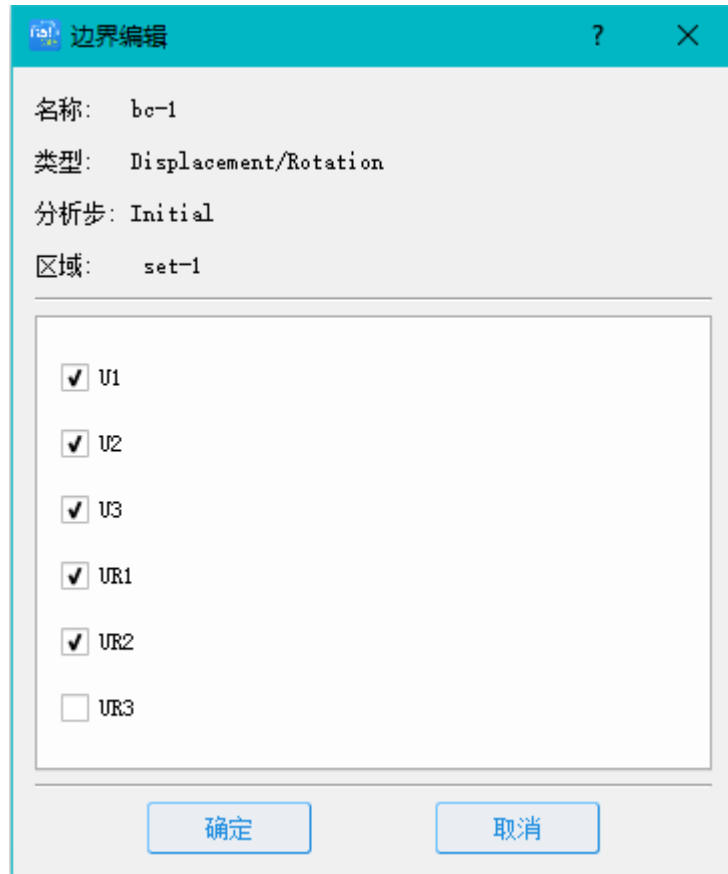


图 25 边界编辑

13. 同样为组件“set-2”设置边界条件。再次点击工程树中的“边界条件”在边界条件创建界面中,名称填写“bc-2”,分析步选择“Initial”,组件选择“set-2”,类别选择“力学”下的“Symmetry/Antisymmetry/Encastre”,点击“继续”。

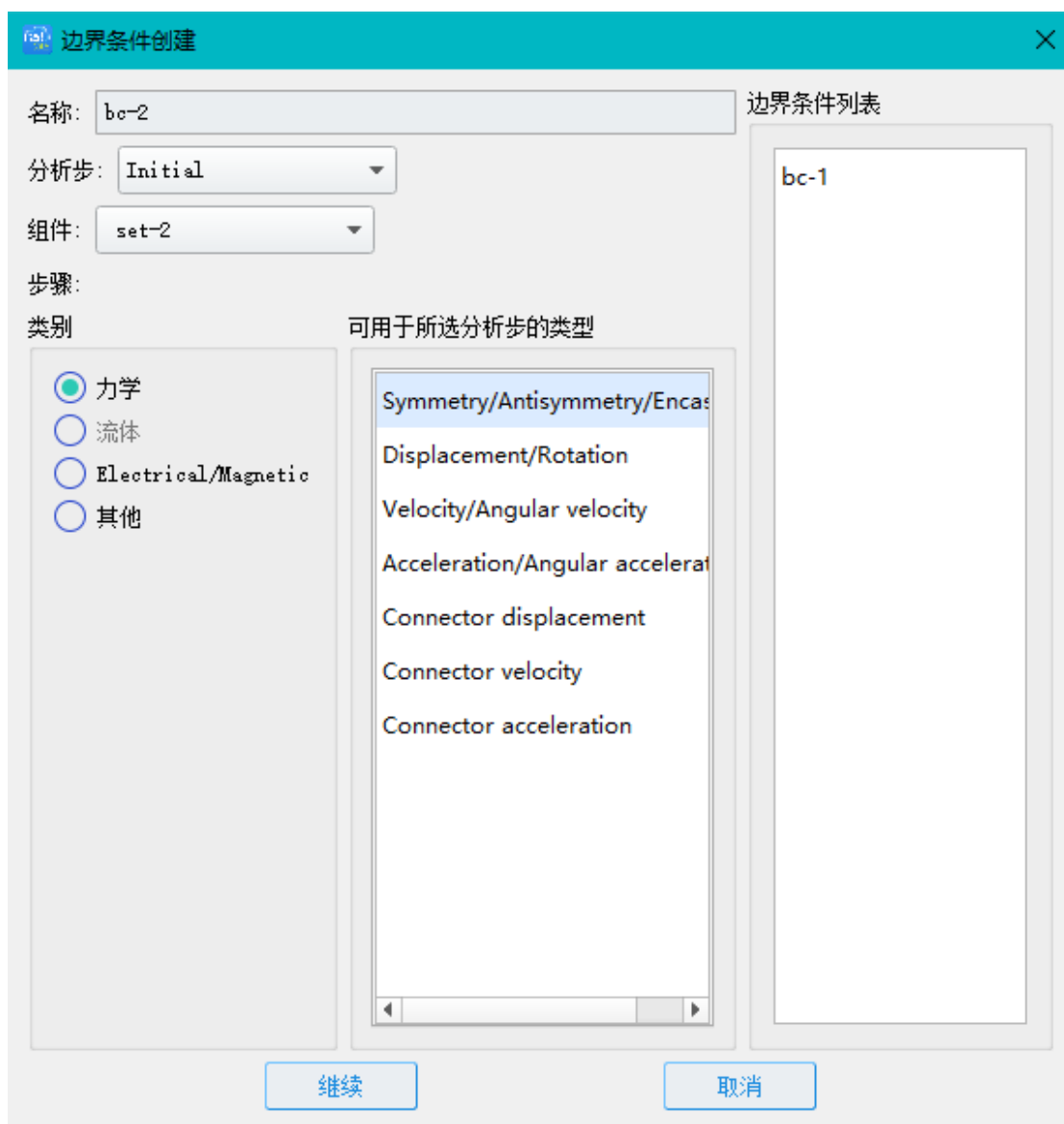


图 26 边界条件创建界面

14. 在边界编辑界面中，选择“ZSYMM”类型，点击“确定”。

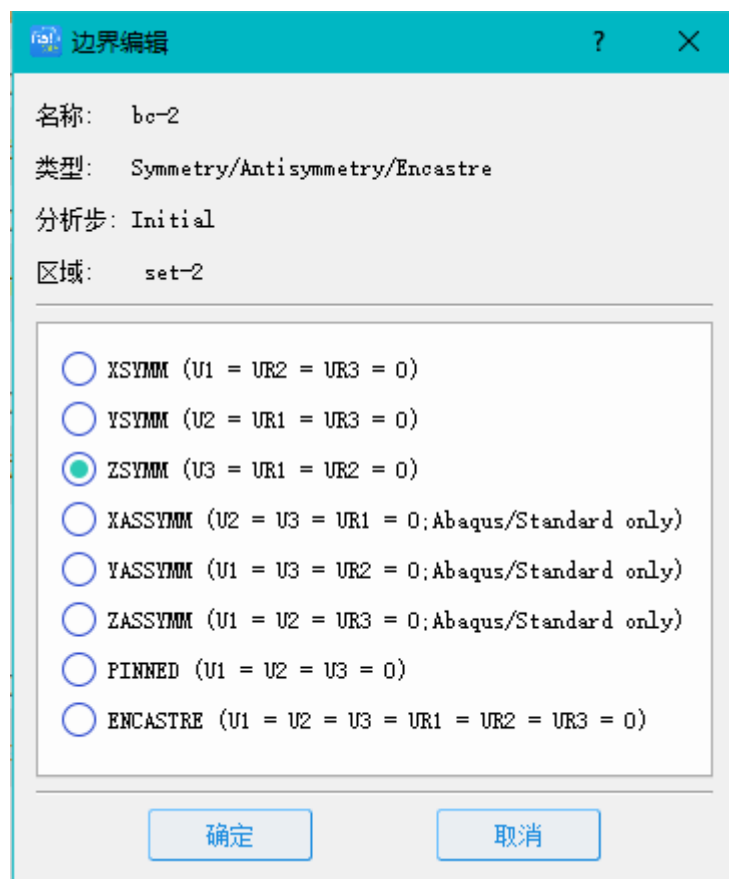


图 27 边界编辑界面

15. 所有需要设置的计算参数设置完毕后，在菜单中点击“求解”下的“求解器管理”设置求解器参数，点击“添加”。

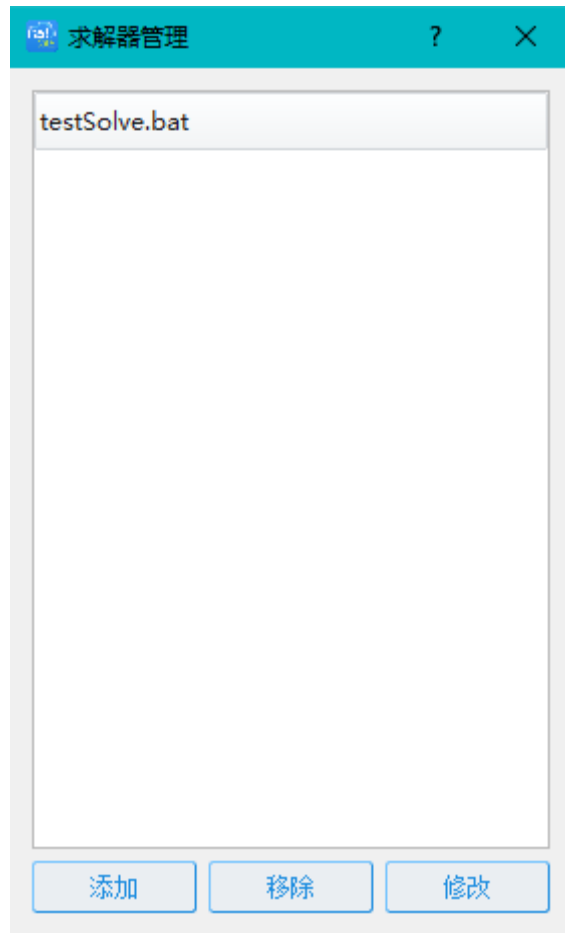


图 28 求解器管理界面



图 29 编辑求解器界面

16. 在“编辑求解器”界面中，设定“求解器类型”为“自研”，求解器名称为“testSolve.bat”，选择该文件所在的路径，“testSolve.bat”文件内容如下：

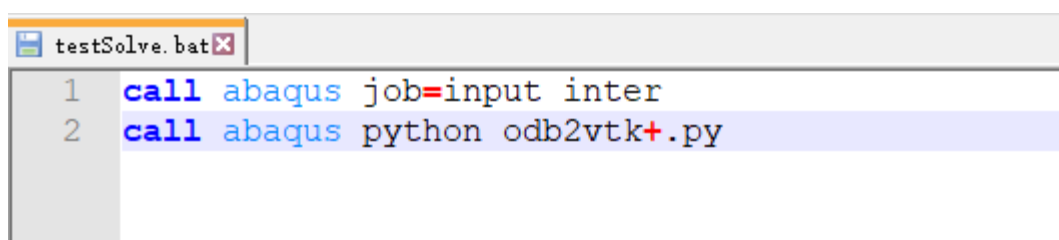


图 30 TestSolve 内容

17. “输入文件”类型选择“文件格式”中的 inp,点击确认及设置完毕求解器参数。

18. 点击工具栏的“求解”按键，软件将启动求解。

19. 求解结束后，选择工程树“矢量”鼠标右键“刷新”。加载后处理文件，如图。

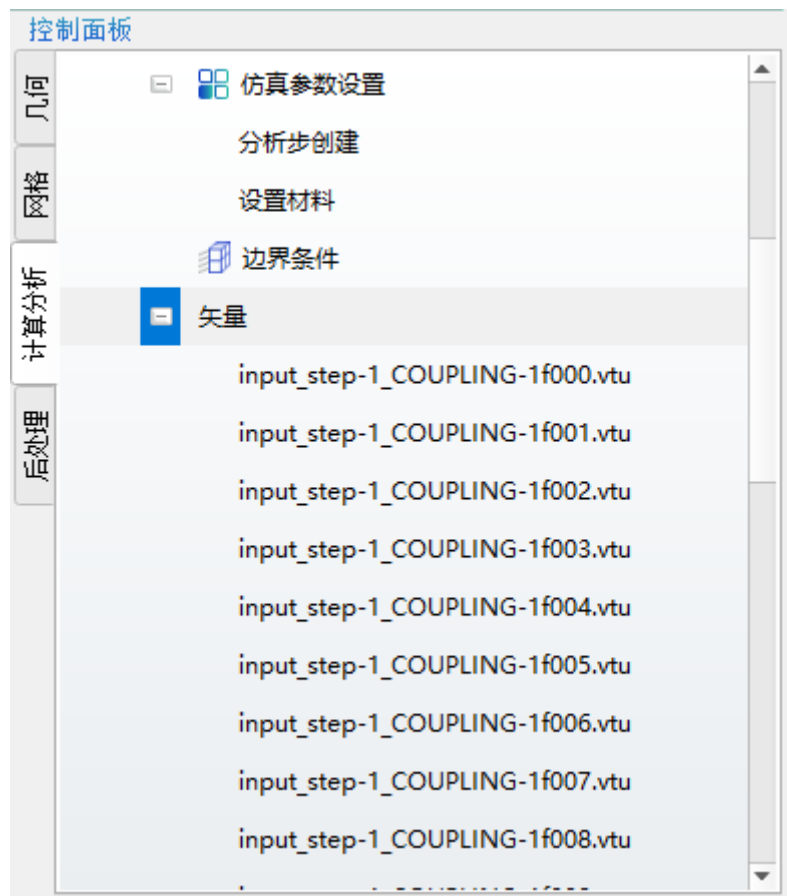


图 31 控制面板界面

20. 点击“矢量”节点下 vtu 文件，可查看矢量三维云图。

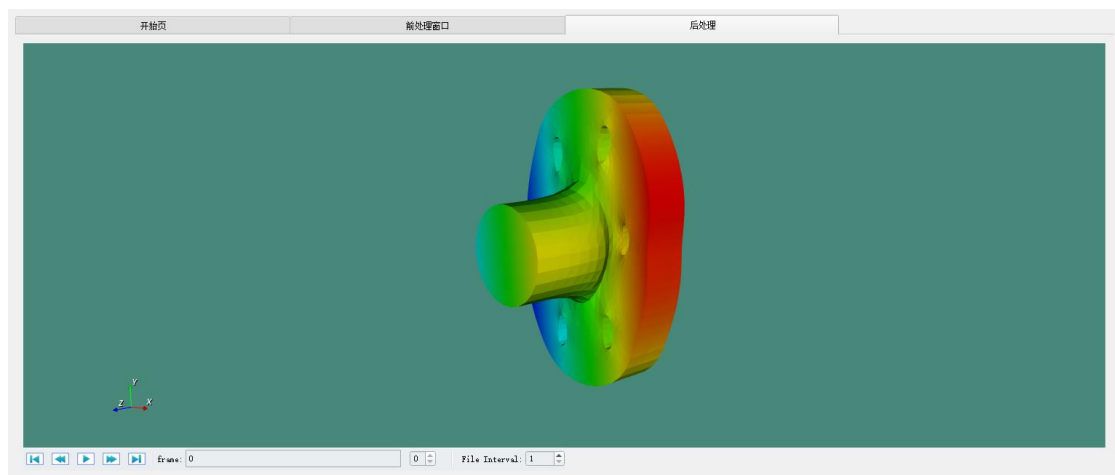


图 32 后处理显示界面